AI Interview Assistant — Project Outline & SOP

Phase 0: Planning document · Personal project · 100% open source, zero cost

1. Project at a glance

The AI Interview Assistant is a web app where a candidate uploads their resume, a job description (JD) and any other supporting documents. The system analyses them, conducts a realistic mock interview tailored to that candidate and that role, and afterwards produces a detailed feedback report.

Hard constraints

- Every component is open source and free to run: LLM, embeddings, speech, database, frontend, backend.
- Backend: FastAPI (Python). Frontend: React.
- Runs locally on the developer's machine (Docker Compose for one-command setup). No paid APIs, no paid hosting.
- Built in phases. Each phase ends with something that works and can be demoed.

2. Goals and non-goals

Goals (v1)

- Accept resume (PDF/DOCX), JD (text/PDF/DOCX) and optional extra documents (project descriptions, company notes, past feedback).
- Extract a structured candidate profile and structured JD requirements, and find the gaps and overlaps between them.
- Run a text-based interview with adaptive follow-up questions.
- Generate a scored feedback report with strengths, weaknesses, per-question analysis and an improvement plan.
- Keep a history of past sessions so progress can be compared over time.

Later goals (v2+)

- Voice interview (speech-to-text and text-to-speech, all local).
- Progress dashboard and trend charts.
- Multiple interview types (technical, HR/behavioural, system design, coding).

Non-goals for v1

- Video or facial/emotion analysis.
- Multi-user accounts, login, billing, multi-tenancy.
- Public cloud deployment (open-source LLMs need a GPU or a strong CPU, which free hosting does not provide).

3. Guiding principles

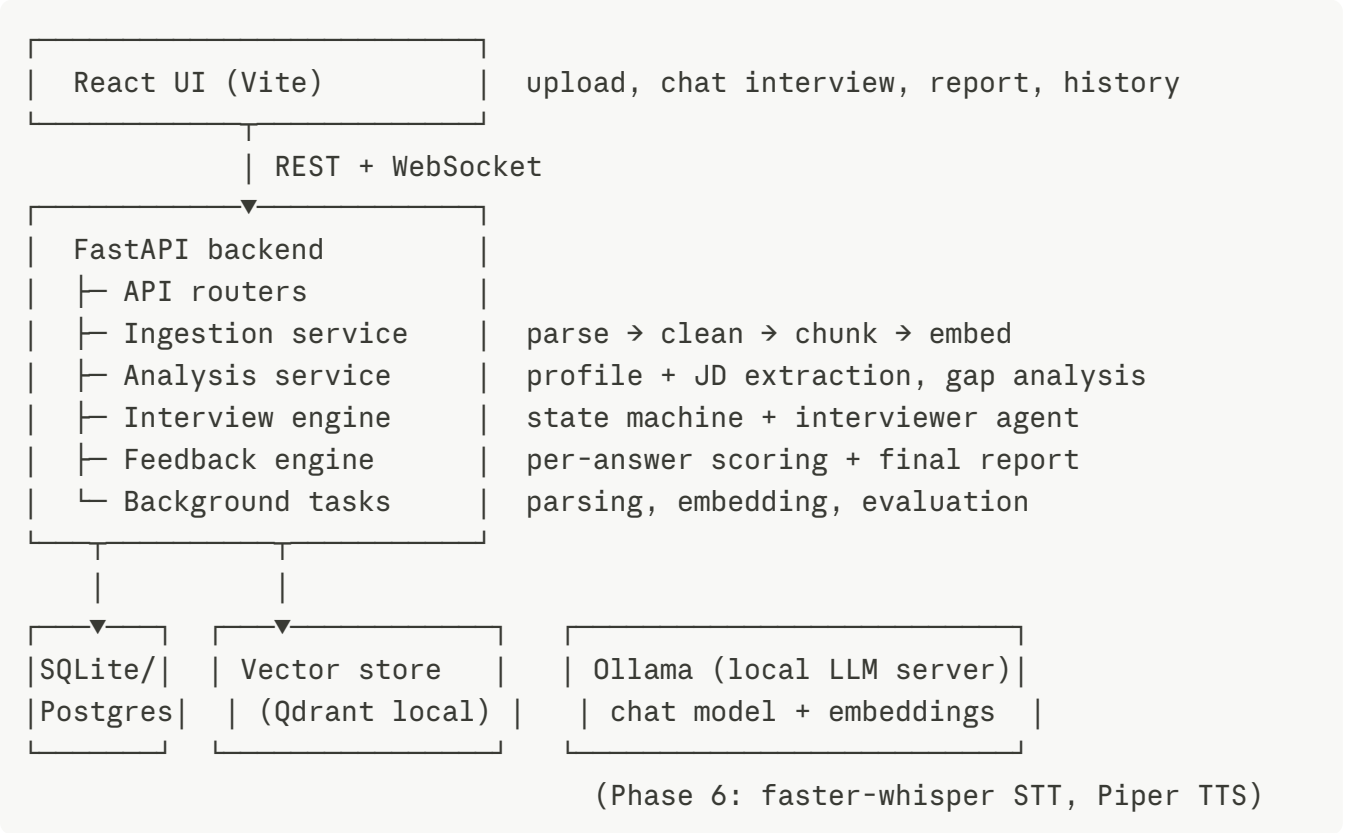
1. **Open source only.** If a tool needs an API key or a credit card, it is out.
2. **Local-first.** The LLM runs on the developer's machine through Ollama. As a bonus, resumes never leave the machine.
3. **Structured outputs everywhere.** The LLM returns JSON that is validated with Pydantic. Free text is only used for what the candidate actually reads or hears.
4. **Small models first.** Design prompts so a 3B–8B model can do the job: short prompts, one task per call, clear rubrics, few-shot examples.
5. **Thin vertical slices.** Get upload → question → answer → feedback working end to end (ugly UI is fine), then improve each stage.
6. **Separate the brain from the plumbing.** The interview logic lives in plain Python modules that can be tested without the web layer.

4. End-to-end user flow

1. **Create a session.** The user gives the session a name, picks the target role, interview type (technical, behavioural, mixed), difficulty and length (for example 8, 12 or 20 questions).
2. **Upload documents.** Resume (required), JD (required), extra documents (optional). The UI shows each document's processing status: Uploaded → Parsing → Ready.
3. **Analysis.** The backend parses the documents, builds a candidate profile and a JD requirement list, and produces a match/gap summary. The user can review and correct it (for example fix a wrongly extracted skill) before moving on.
4. **Interview plan.** The system generates a question plan: topics to cover, depth per topic and a rough order. The user never sees the full question list, to keep the interview realistic, but sees the topic outline.
5. **Live interview.** The interviewer agent asks one question at a time. After each answer it decides: ask a follow-up, probe deeper, move to the next topic, or wrap up. Every answer is saved and queued for evaluation.
6. **Feedback.** When the interview ends, the evaluator produces a report: overall score, score per skill area, per-question analysis with a stronger sample answer, communication feedback and a concrete improvement plan.

7. **History.** Past sessions and reports are stored and can be reopened or compared.

5. System architecture



Responsibilities

- **React UI** only renders state and sends user input. No interview logic lives here.
- **FastAPI** exposes the API, validates input, owns the database and orchestrates the engines.
- **Ollama** hosts the open-source models behind a simple local HTTP API.
- **Vector store** holds embedded chunks of the resume, JD and extra documents so questions and evaluations can be grounded in the candidate's real content.
- **Relational DB** holds sessions, documents, profiles, questions, answers, evaluations and reports.

6. Technology stack

All of the following are open source and free. Alternatives are listed so a swap is easy.

Layer	Choice	Why	Alternatives
Backend	FastAPI + Uvicorn	Async, typed, automatic Swagger docs	—
Validation	Pydantic v2	Schemas for API and LLM outputs	—
Frontend	React + Vite + TypeScript	Fast dev loop, large ecosystem	Next.js
UI styling	Tailwind CSS + shadcn/ui	Quick, clean components	Material UI
LLM runtime	Ollama	One-command local model serving, JSON/structured output support	llama.cpp server, vLLM (needs GPU)
Embeddings	nomic-embed-text or bge-small via Ollama / sentence-transformers	Small, good quality, CPU-friendly	e5-small
Vector DB	Qdrant (embedded local mode)	Free, runs inside the Python process (no server or Docker needed), and the same API works against a Qdrant server later	ChromaDB, pgvector, FAISS
Relational DB	SQLite for dev, PostgreSQL for later	Zero setup first	—
ORM / migrations	SQLAlchemy 2 + Alembic	Standard with FastAPI	SQLModel
Document parsing	PyMuPDF, pdfplumber, python-docx	Reliable text extraction with fallback	Unstructured
RAG glue	LangChain (loaders, text splitters, retrievers, Qdrant and Ollama integrations)	Ready-made integrations keep the code short; the interview state machine itself stays plain Python	Plain Python, LlamaIndex

Layer	Choice	Why	Alternatives
Background work	FastAPI BackgroundTasks first; Celery + Redis when needed	Start simple	RQ, Arq
Real-time chat	WebSocket (or Server-Sent Events for streaming tokens)	Streaming feels like a live conversation	—
Speech-to-text (Phase 6)	faster-whisper	Fast, accurate, runs locally	whisper.cpp, Vosk
Text-to-speech (Phase 6)	Piper	Light, offline, natural enough	Coqui TTS
Packaging	Docker Compose	One command to start everything	—
Testing	pytest, Vitest / React Testing Library	—	—

Note on hosting. Free cloud tiers cannot run an LLM. The honest plan is local-first. If it ever needs to be shared, a small demo can be recorded as a video, or the backend can run on a personal machine and be exposed temporarily with a free tunnel tool.

7. Choosing the open-source LLM

The model choice is mostly a hardware question. Check what the machine has before deciding.

Your machine (16 GB RAM, 250 GB storage). Target 7B–8B models at 4-bit quantisation. Each one needs roughly 5–6 GB of RAM, so run one chat model plus one small embedding model at a time and keep the browser and other heavy apps closed during interviews. Without a dedicated GPU, generation is slow (typically single-digit tokens per second), so token streaming in the UI is essential. A 12B–14B model for the evaluator is possible but leaves little headroom, so test it before committing. Storage is not a constraint: each 7B–8B model is about 4–5 GB, so keeping 2–3 candidates for the Phase 2 benchmark is fine.

Machine	Suggested size class	Notes
8 GB RAM, no GPU	3B–4B models, 4-bit quantised	Works but slow; keep prompts short

Machine	Suggested size class	Notes
16 GB RAM or 8 GB VRAM	7B–8B models, 4-bit quantised	Best balance for this project
24 GB+ VRAM	12B–14B+ models	Noticeably better evaluation quality

Model families to evaluate (all available through Ollama): Llama 3.x, Qwen 2.5 / newer Qwen releases, Mistral, Gemma, Phi. Newer versions appear often, so check the Ollama library for the current best model in each size class before Phase 2 and compare two or three on the same test prompts.

Use different models for different jobs

- **Interviewer model:** needs to be fast and conversational. A 7B–8B model is enough.
- **Evaluator model:** quality matters more than speed because it runs after the interview. Use the largest model the machine can handle. It is fine for feedback to take a minute.
- **Embedding model:** a small dedicated embedding model, not the chat model.

Selection method: build a small benchmark (5 resumes × 3 JDs, plus 20 sample answers of varying quality) and compare models on JSON validity rate, question relevance and scoring consistency. Choose by evidence, not reputation.

8. How each core piece works

8.1 Document ingestion

1. Accept PDF, DOCX and TXT. Validate type and size.
2. Extract text. For PDFs try PyMuPDF first, fall back to pdfplumber. Keep page and section boundaries where possible.
3. Clean the text: normalise whitespace, remove headers/footers, fix broken bullet characters.
4. Detect the document role (resume, JD, other) from the user's upload choice, not from guessing.
5. Split into chunks by section heading (Experience, Projects, Skills) rather than blind character counts. Resumes are short, so section-level chunks work better than 4,000-character chunks.
6. Embed chunks and store them with metadata: session_id, document_id, section, source.
7. Store the raw text in the relational DB as well, since the resume and JD are short enough to pass whole into prompts.

Design note. A resume and a JD usually fit entirely in the context window. Use RAG for long or numerous extra documents. Use full-text injection for the core resume and JD. This avoids retrieval misses on the most important inputs.

8.2 Profile and JD analysis

Two extraction calls, each returning validated JSON.

Candidate profile: name, total experience, current role, skills (with an estimated level and evidence), employers and durations, projects (title, tech stack, claimed outcome), education, certifications.

JD requirements: role title, seniority, must-have skills, nice-to-have skills, responsibilities, domain, soft skills.

Gap analysis (a third call, or plain code over the two JSON objects):

- Skills that match (candidate has it, JD wants it).
- Gaps (JD wants it, candidate does not show it).
- Claims worth probing (impressive numbers, vague ownership, buzzwords with no detail).
- Candidate strengths to use as warm-up topics.

Show this result to the user for review. It builds trust, and it fixes extraction mistakes before they affect the interview.

8.3 Interview plan

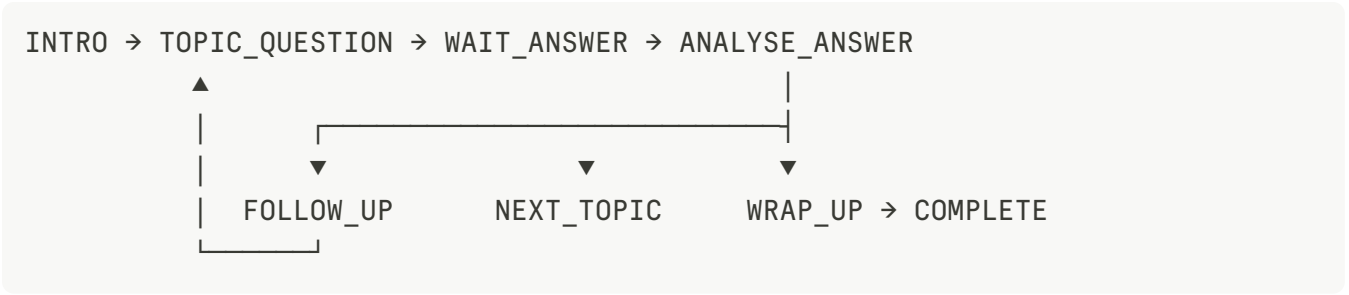
From the gap analysis and the user's settings, generate a plan such as:

- Intro and motivation (1 question)
- Core skills from the JD (40% of the questions)
- Deep dive on 1–2 resume projects (30%)
- Gap areas, asked at a fair difficulty (15%)
- Behavioural / situational (15%)
- Candidate's questions for the interviewer (1 question)

Each plan item has a topic, a goal (what the interviewer wants to learn), a difficulty and a maximum number of follow-ups. The plan is data, so it can be inspected, tested and adjusted.

8.4 Interview engine

A small, explicit state machine. The LLM writes the words, but the code controls the flow.



After every answer, one quick "analyse" call returns JSON:

- answer_quality (weak / ok / strong)
- missing_points
- is_off_topic
- needs_follow_up (true/false)
- suggested_follow_up

Rules enforced in code, not left to the model:

- A maximum number of follow-ups per question (for example 2).
- Total question count and rough time budget.
- No repeating a question already asked.
- Never reveal evaluation or scores during the interview.
- If the model returns invalid JSON, retry once, then fall back to the next planned question.

Interviewer persona. One system prompt: professional, neutral, one question at a time, short acknowledgements ("Thanks, let's move on"), no coaching or hints during the interview.

Conversation memory. Keep the last few turns plus a running summary rather than the whole transcript, so small models stay fast and focused.

8.5 Feedback engine

Step 1: per-answer evaluation (runs in the background during the interview, or in a batch after it). Each answer is scored against a rubric on a 1–5 scale with anchored descriptions:

Dimension	What it measures
Technical accuracy	Is the content correct?
Depth	Did the candidate go beyond definitions into trade-offs and real experience?
Relevance	Did they answer the question that was asked?

Dimension	What it measures
Structure and clarity	Logical order; STAR format for behavioural answers
Evidence	Concrete examples, numbers and ownership (“I” versus “we”)

Each score must come with a short evidence quote from the answer. This stops the model from giving vague, inflated scores.

Step 2: final report (one call over the per-answer results plus the profile and JD):

- Overall score and a hire-readiness style verdict for the target role.
- Scores per skill area and per rubric dimension.
- Top 3 strengths and top 3 improvement areas.
- Per-question breakdown: question, answer summary, score, what was missing, a stronger sample answer.
- Communication notes: answer length, filler or vague phrases, rambling, one-line answers.
- A 1–2 week improvement plan with specific topics to study.

Known weakness: small models are generous judges. Reduce this with anchored rubrics, low temperature, evidence quotes, a stronger model for evaluation than for interviewing, and a few reference answers of known quality to calibrate against.

8.6 Prompting rules

- One task per call. Do not ask one prompt to analyse, score and rewrite.
- Request JSON that matches a Pydantic schema. Use Ollama’s structured output / JSON format option, validate the result, retry once on failure.
- Low temperature (0–0.2) for extraction and evaluation, moderate (0.5–0.7) for interviewer wording.
- Put resume and JD content inside clearly delimited blocks in the prompt, and tell the model that text inside them is data, not instructions. This reduces prompt-injection problems from uploaded files.
- Keep all prompts in versioned files (`prompts/`), not inline in code, so they can be iterated and tested.

9. Data model (first draft)

Table	Key fields
sessions	id, title, target_role, interview_type, difficulty, question_count, status, created_at
documents	id, session_id, kind (resume / jd / other), filename, raw_text, status
candidate_profiles	id, session_id, profile_json
jd_profiles	id, session_id, requirements_json
gap_analyses	id, session_id, analysis_json
interview_plans	id, session_id, plan_json
questions	id, session_id, order_no, topic, text, is_follow_up, parent_question_id
answers	id, question_id, text, duration_seconds, created_at
evaluations	id, answer_id, scores_json, evidence_json, feedback_text
reports	id, session_id, overall_score, report_json, created_at

Vector store: one collection per session (or one collection with a session_id filter), holding document chunks and their metadata.

10. API outline

Method	Endpoint	Purpose
POST	/sessions	Create a session with settings
GET	/sessions, /sessions/{id}	List and view sessions
POST	/sessions/{id}/documents	Upload resume, JD or extra documents
GET	/sessions/{id}/documents	Processing status per document
POST	/sessions/{id}/analyze	Build profile, JD requirements and gap analysis
GET / PATCH	/sessions/{id}/analysis	Read and correct the analysis

Method	Endpoint	Purpose
POST	/sessions/{id}/plan	Generate the interview plan
WS	/sessions/{id}/interview	Live interview: send answers, receive questions (streamed)
POST	/sessions/{id}/finish	End the interview early and trigger feedback
GET	/sessions/{id}/report	Feedback report (with status while it is generating)
DELETE	/sessions/{id}	Delete a session and all its data

FastAPI's automatic Swagger page doubles as living API documentation.

11. Frontend screens

1. **Home / history:** past sessions with scores and a "New interview" button.
2. **Setup:** role, interview type, difficulty, number of questions.
3. **Upload:** drag-and-drop for resume, JD and extras, with per-file status.
4. **Analysis review:** extracted skills, JD requirements, match and gap summary, editable.
5. **Interview room:** chat-style view, streaming interviewer messages, answer box, progress indicator, optional timer and an "End interview" button. Voice controls arrive in Phase 6.
6. **Report:** score summary, radar or bar chart per skill area, expandable per-question cards, improvement plan, export to PDF.
7. **Settings:** choose the Ollama model and see connection status.

12. Proposed repository structure

```
ai-interview-assistant/
├─ backend/
│  ├─ app/
│  │  ├─ main.py
│  │  ├─ api/            # routers: sessions, documents, interview, report
│  │  ├─ core/           # config, logging, db setup
│  │  ├─ models/         # SQLAlchemy models
│  │  ├─ schemas/        # Pydantic schemas (API + LLM outputs)
│  │  ├─ services/
│  │  │  ├─ ingestion/   # parsers, cleaner, chunker, embedder
│  │  │  ├─ analysis/    # profile, JD, gap analysis
│  │  │  ├─ interview/   # planner, state machine, interviewer
```

```
|  |  |  ├─ feedback/    # evaluator, report builder
|  |  |  └─ llm/         # Ollama client, JSON retry helper
|  |  └─ prompts/        # versioned prompt templates
|  ├─ tests/
|  └─ pyproject.toml
├─ frontend/
|  └─ src/ (pages, components, api client, hooks)
├─ data/samples/         # sample resumes, JDs, benchmark answers
├─ docs/                 # this SOP, architecture notes, decisions
├─ docker-compose.yml
└─ README.md
```

13. Phased roadmap

Time estimates assume part-time, personal-project pace. Treat them as rough.

Phase 0 — Planning (this document)

- **Output:** agreed scope, stack, flow, data model and roadmap.
- **Exit criteria:** open decisions in section 16 answered; repository created with a README.

Phase 1 — Foundation and document ingestion (about 1 week)

- FastAPI project, config, SQLite database, Docker Compose skeleton.
- Session and document endpoints; file upload and validation.
- PDF/DOCX/TXT parsing with fallback; cleaning and section detection.
- Basic React shell with the upload screen.
- **Exit criteria:** upload a resume and a JD, see clean extracted text and status in the UI. Parser tests pass on at least 5 sample resumes.

Phase 2 — LLM integration and analysis (about 1 week)

- Ollama client with JSON-schema output, validation and retry helper.
- Candidate profile and JD extraction; gap analysis.
- Embeddings and vector store for extra documents.
- Model benchmark script; pick the interviewer and evaluator models.
- Analysis review screen.
- **Exit criteria:** profile and JD JSON is valid on at least 95% of benchmark inputs; the gap summary reads sensibly on manual review.

Phase 3 — Interview plan and text interview (1–2 weeks)

- Planner generating the topic outline.
- Interview state machine, interviewer prompts, follow-up logic, per-answer analysis.
- WebSocket chat with token streaming; interview room UI.
- Persist questions and answers.
- **Exit criteria:** a full 10-question interview runs end to end without repeating questions or breaking the flow; follow-ups feel relevant.

Phase 4 — Feedback and report (1 week)

- Rubric-based evaluator with evidence quotes.
- Final report generation and the report UI with charts.
- Calibration with known strong and weak sample answers.
- **Exit criteria:** strong sample answers score clearly higher than weak ones; the report is specific to the candidate, not generic.

Phase 5 — History, polish and hardening (about 1 week)

- Session history, comparison between attempts, PDF export.
- Error handling for Ollama being down or slow, upload limits, delete-my-data.
- Move long jobs to Celery/Redis only if BackgroundTasks proves insufficient.
- Test coverage on the engines; README, screenshots and a demo video.
- **Exit criteria:** a new person can clone the repository, run Docker Compose and complete an interview following only the README.

Phase 6 — Voice mode (optional, 1–2 weeks)

- Browser audio capture, faster-whisper transcription, Piper text-to-speech.
- Speaking-pace and filler-word metrics added to the report.
- **Exit criteria:** a spoken interview works with acceptable latency (aim for a few seconds per turn).

Phase 7 — Stretch ideas

- Coding-round mode with a sandboxed code runner.
- System design round with diagram upload.
- Company-specific question banks from user-provided notes.
- Progress dashboard over many sessions.

14. Testing and quality

LLM apps fail quietly, so quality needs its own tests.

- **Unit tests:** parsers, chunker, state machine transitions, JSON repair/retry, score aggregation.
- **Golden tests:** fixed sample resumes and JDs with expected extracted skills; the test checks that the key skills are found.
- **LLM output checks:** schema validity rate, duplicate-question rate, off-topic rate.
- **Evaluator calibration:** a small set of answers labelled strong, average and weak by hand. The evaluator must rank them correctly.
- **Latency budget:** time to first token for a question, time to generate the report. Track them per model.
- **Manual review:** after each phase, run one full interview yourself and write down what felt wrong.

15. Risks and mitigations

Risk	Mitigation
Small model gives weak or repetitive questions	Planner-driven topics, “already asked” list in the prompt, benchmark several models, allow a larger model for evaluation
Invalid or inconsistent JSON	Structured output mode, Pydantic validation, one retry, safe fallback
Inflated or inconsistent scoring	Anchored rubric, evidence quotes, temperature near 0, calibration set
Slow responses on CPU	Stream tokens, smaller interviewer model, short prompts, run evaluation in the background
Hallucinated candidate facts in questions	Include the resume text in the prompt and ask only about stated content; show the analysis for user correction
Messy or scanned PDFs	Parser fallback chain; optional OCR (Tesseract) as a later addition
Prompt injection from uploaded files	Delimit documents, instruct the model to treat them as data, never let model output trigger actions

Risk	Mitigation
Scope creep (voice, coding rounds, dashboards)	Strict phase gates; the optional features wait until Phase 5 is done
Privacy of resumes	Fully local processing, delete-session endpoint, no telemetry

16. Decisions to make before Phase 1

1. **Hardware:** decided: a 16 GB RAM laptop with 250 GB storage, so the target is 7B–8B quantised models.
2. **Interview type for v1:** technical only, or technical plus behavioural? (Recommendation: mixed, with technical as the main part.)
3. **Target domain:** decided: Python / AI roles first, then generalise to other roles after v1.
4. **Vector store:** decided: Qdrant in embedded local mode (free, no server needed).
5. **RAG framework:** decided: LangChain for loaders, splitters, embeddings and retrievers; the interview flow stays plain Python.
6. **Voice:** confirm it is Phase 6 and not part of v1.
7. **Language:** decided: English only.
8. **Repository:** name, licence (MIT is a common choice for a personal open-source project), public or private.

17. Next step

Hardware, target domain, vector store, RAG framework and language are settled. Still open in section 16: interview type, voice timing, and the repository name and licence. After that, Phase 1 begins with the repository scaffold, Docker Compose file and the document upload and parsing pipeline. A detailed Phase 1 task breakdown can be written as the next SOP.